

MEDHAVA

What to buy and switch on

8 steps, in the order they should happen. Each says what it is for, why it sits where it does, and how you know it worked.

Nothing here quotes a price as a fact. What each option costs changes, and a stale price in a document is worse than no price because somebody plans around it. Where a figure appears it is a range from the project's own tools register and should be checked on the day. I cannot reach any of these providers' websites from where this was written, so nothing below claims to describe their current offering.

Most of this list is free. The first step costs nothing, and the one item with no free path at all is last on purpose.

1 · Put the code somewhere both you and every AI tool can reach

A private repository holds the whole project — the code, the registers, the tests and the gates. This is the single most important step and it is free. Everything after it assumes the code is somewhere other than one laptop.

Why it is at number 1. First, because it is what makes the rest survivable. A chat window closes and a subscription runs out; a repository does neither. It is also the answer to "how do I carry this to Codex" — Codex opens the same folder and runs the same checks.

What this is for	Source control and continuous integration
The free option	GitHub free — unlimited private repositories, 2,000 Actions minutes a month
The paid option	GitHub Team, about \$4 per user a month
When paying becomes worth it	past 2,000 CI minutes a month, or when branch protection across a team is needed

What to do

- Create a private repository and push this project into it
- Confirm the checks run there — the workflow in `.github/workflows/ci.yml` runs on push

You know it worked when: You can clone the project onto a machine that has never seen it, run `npm ci` and then `npm test`, and it passes.

2 · Decide the names before you buy the machine

Four names, all pointing at one machine, each serving something different: the public site, the application behind sign-in, and whatever you use for internal tools. Mail stays separate and points at whoever provides your mailboxes.

Why it is at number 2. Before the machine, because the names decide how it is configured and because DNS changes take time to spread. Deciding them afterwards means redoing the web server and the certificates.

What to do

- Write down which name serves what, using the table in DEPLOYMENT.md section 4
- Leave the records unpointed until the machine exists — you already own the domain

You know it worked when: You have four names written down and know which one serves the public site and which one serves the application.

Detail: `DEPLOYMENT.md`

3 · The machine everything else runs on

One server that runs the application. A VPS is a computer you rent and administer yourself — more work than a managed host and much cheaper, and it is the only option of the two that can also run the automation and the local AI model later.

Why it is at number 3. Third, because steps 4 to 7 all run ON it. Buying it earlier means paying for an idle machine; buying it later blocks everything else.

What this is for	Hosting the web application
The free option	Vercel Hobby, Cloudflare Pages or Netlify free tier — enough for a pilot and a demo
The paid option	Vercel Pro, about \$20 a month per member
When paying becomes worth it	commercial use under their terms, or a team that needs shared deploys and password-protected previews

What to do

- Rent the machine

- Follow DEPLOYMENT.md sections 2 and 3 before anything listens on the internet — secure it first, then give it room
- Point the names from step 2 at it, then get certificates (sections 4 and 5)

You know it worked when: Each name resolves to the machine, checked from a connection that is not yours, and the site answers over https.

Detail: [DEPLOYMENT.md](#)

4 · The database, and the role the application connects as

Where every business record lives. The part that matters more than the choice of provider is the ROLE: the application must connect as a user that is neither the superuser nor the owner of the tables.

Why it is at number 4. Immediately after the machine, because nothing can be stored before it exists — and because getting the role wrong at the start is expensive to undo once there is data.

What this is for	Database, auth and row-level security
The free option	Supabase free tier — Postgres 16, 500 MB, 50k monthly active users, daily backups
The paid option	Supabase Pro, about \$25 a month
When paying becomes worth it	past 500 MB of data or 50k monthly active users, or when point-in-time recovery is needed

What to do

- Create the database and load core/schema.postgres.sql
- Create the application role exactly as DEPLOYMENT.md section 6a sets out
- Run `node core/tests/live.test.js` against it

You know it worked when: That test passes against your real database. It is the one check that proves one company cannot read another company’s rows — and it only proves it because the role is restricted. As a superuser it would pass while proving nothing.

Detail: [DEPLOYMENT.md](#)

5 · Know when it breaks before a customer tells you

Two separate things: something that tells you the site stopped answering, and something that records errors with enough detail to fix them.

Why it is at number 5. Before real users, not after. An outage you hear about from a customer has already cost you the thing monitoring was meant to protect.

What this is for	Error tracking and uptime
The free option	Sentry developer tier (5k errors a month) and any free uptime pinger
The paid option	Sentry Team, about \$26 a month
When paying becomes worth it	past 5k errors a month — which usually means something is wrong that should be fixed rather than paid for

What to do

- Set up the health check described in DEPLOYMENT.md section 11
- Point an uptime checker at it
- Confirm backups run, and restore one — a backup nobody has restored is a hope

You know it worked when: You deliberately stop the application and receive an alert without anyone telling you.

Detail: [DEPLOYMENT.md](#)

6 · Automation, on the machine you already rent

Scheduled jobs and small workflows — a nightly report, a reminder, a file picked up and processed. Self-hosted on the same VPS rather than a monthly subscription.

Why it is at number 6. After the machine and the database, because most useful automations read or write business data. Before the AI work, because several AI jobs are triggered by it.

What this is for	Automations and scheduled jobs
The free option	n8n self-hosted on the VPS, or plain cron plus a webhook
The paid option	n8n Cloud from about €20 a month, or Make/Zapier on usage
When paying becomes worth it	only when nobody can maintain a VPS — the self-hosted version is the same software, not a cut-down one

What to do

- Install it on the VPS from step 3
- Put it behind sign-in and never expose it publicly
- Start with one real job you currently do by hand, not with a demo

You know it worked when: One job you used to do by hand now runs on its own, on schedule, and you have stopped doing it — not once as a demonstration, but for a week without you touching it.

Detail: [MEDHAVA_BUILD_GUIDE.md](#)

7 · A model on your own machine, before a model you pay per call for

A small model running on the same VPS. Free, private — no business data leaves the machine — and good enough for summarising, classifying and drafting.

Why it is at number 7. After automation, because that is what will call it. Before any paid AI account, because you cannot tell whether you need one until you have seen what the free option does on your own questions.

What this is for	AI for writing, summarising and the assistant
The free option	Ollama running a small model on the same VPS or a laptop — free, private, and good enough for summarising and classifying
The paid option	A hosted model API, billed per token
When paying becomes worth it	when answer quality on real business questions is not good enough locally, or the machine cannot hold the model

What to do

- Install it on the VPS and run one real task through it
- Judge it on your own data, not on a demo
- Move to a paid model only when the trigger in tools.js is actually met

You know it worked when: It answers a real question about your own business well enough to use, or you have established that it does not — either is a result.

Detail: [MEDHAVA_ARCHITECT.md](#)

8 · WhatsApp — the one thing on this list with no free path

Business messaging goes through an approved provider. There is no free tier: the provider charges a monthly fee and the conversations are charged separately at rates the platform owner sets, not the provider.

Why it is at number 8. Last of the running costs, and only when somebody is actually waiting to use it. It is the first item here that bills you every month whether or not it is used, so it should start the day a worker or a customer is expected on it and not before.

What this is for	WhatsApp for the shop floor and customers
The free option	None. There is no free path to WhatsApp Business messaging.
The paid option	A Business Solution Provider — Interakt, Wati, AiSensy and similar, roughly ₹1,500–3,000 a month plus per-conversation charges set by Meta
When paying becomes worth it	the first day a worker or a customer is expected to use WhatsApp — there is no free tier to outgrow

What to do

- Choose a provider and complete their business verification — this takes days, not minutes, so start it before the day you need it
- Connect it to the automation from step 6

You know it worked when: A message sent from the shop floor becomes a real record in the system, and the person who sent it can see it worked.

Detail: `MEDHAVA_PLAN_OF_ACTION.md`

What is deliberately not on this list

Anything that bills you before the thing it pays for can be used. The whole register of 19 capabilities is in `brand/site/tools.js`, and the build refuses any paid choice there that does not also name a free option and the point at which paying becomes worth it. That refusal is what keeps this list short.

Every technical word above, in plain language

16 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

backup

A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.

Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

job

One piece of work taken off the queue and done in the background.

Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

continuous integration

A robot that checks every change automatically, before anyone can put it live.

Quality-check wala banda gate pe khada. Har maal nikalne se pehle usse guzarta hai.

uptime

How much of the time the system is actually working and reachable.

Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.

model

The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.

Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.